



AI Engineer — Technical Assessment Round

Thank you for making it through the first round with us. This assessment is designed around the kind of AI engineering work our team does day to day — building retrieval-augmented, agentic systems on top of imperfect real-world documents, under real production constraints.

Requirements:

1. You have 4–5 calendar days to complete this from the day you receive it.
2. We expect roughly 10–14 hours of total effort, assuming you're working around a full-time job.
3. Python is required for this assessment, in line with the frameworks and libraries used day to day in your current role.
4. If a piece is genuinely out of scope for the time budget, say so explicitly in your design doc rather than skipping it silently. We'd rather see your reasoning than a rushed implementation.

Scenario:

SunGrid Cooperative is a fictitious solar energy cooperative. Members interact with a support team that answers questions using an internal knowledge base spanning program policies, incentive/rebate programs, billing FAQs, technical/installation guidance, and company updates.

SunGrid wants a prototype AI Copilot that can answer member questions accurately by retrieving from this internal knowledge base — and, in some cases, needs to call a separate eligibility-check service rather than answer from documents alone.

A starter kit is attached separately containing:

- docs/ — 14 synthetic knowledge-base documents across five categories (one deliberately ambiguous — see Part A)
- stub_tools.py — a mock eligibility-check function standing in for a downstream service that isn't available yet
- README.md — a short pointer file



Part A — Ingestion, Classification & Retrieval

Build an ingestion pipeline over the provided documents (chunking + embedding + a vector store of your choice).

Build a classifier that assigns each incoming query to a single primary category (e.g. Program Policies, Incentive & Rebate Programs, Billing & Account, Technical/Installation Guidance, Company Updates, or non_relevant), and use that category as a retrieval filter — not as a top-level routing decision that changes your whole pipeline.

Note: one of the fourteen documents (rebate refund / billing adjustment) deliberately doesn't sit cleanly under a single category. We want to see how you reason about — and document — that ambiguity, not necessarily a "correct" answer.

Part B — Agent & Tool Use

Build a minimal agent (2+ steps) that, for eligibility-related queries, calls the provided `check_rebate_eligibility` tool rather than answering from retrieved text — and for general knowledge questions, retrieves and composes an answer as in Part A.

- Keep the eligibility tool call as a distinct step/component from your retrieval logic, not folded inside it.
- The eligibility check should behave as a hard gate — a query that fails any eligibility condition should not receive a partial or estimated answer that ignores the failure.
- Treat `stub_tools.py` as a fixed interface representing a service someone else owns. Wire into it; don't rewrite its internals.

Part C — Design Write-Up (no code required)

In 1–2 pages, address:

- How would you evaluate this system before shipping it (offline evaluation and/or online monitoring)?
- What breaks first at 10x the document volume or 100x the query volume, and what would you change?
- If SunGrid added a brand-new document category next month, what in your design would need to change vs. stay the same?



- Name one non-obvious trade-off you made while building this, and why you made it.

Deliverables

- Code: a public GitHub repository link, properly structured (clear folder layout, requirements.txt or equivalent, no committed API keys or credentials).
- README: how to install, configure, and run your solution end to end, plus any assumptions you made.
- Design doc: 1–2 pages (PDF or Markdown, can live in the repo), covering Part C.
- Once you submit your repo link, we'll schedule a short demo and Q&A session where you walk us through your design and decisions. This is a conversation, not a formal presentation.

You're responsible for your own LLM API access for this assessment (OpenAI, Anthropic, Cohere, Google, or an open-source model via Hugging Face all have usable free tiers). Do not commit any API key to your repository — use environment variables and document what's required in your README.

If anything in this brief is ambiguous, treat that ambiguity the way you would on the job: make a reasonable assumption, document it clearly, and move forward. If something seems truly blocking, reach out and we'll clarify quickly.

Good luck — we're looking forward to seeing how you think through this.